

COLLEGAMENTO STATICO E COLLEGAMENTO DINAMICO

I programmi sono formati da diversi elementi: ci sono le variabili, le strutture di controllo e, nei linguaggi orientati ad oggetti, anche gli oggetti e i metodi. Questi elementi hanno significato solo se vengono collegati in un modo opportuno. È quindi necessario che le variabili vengano collegate con il loro tipo e allo stesso modo i metodi vengano collegati con l'oggetto al quale si riferiscono.

Esistono due tipi di collegamento (**binding**) possibili:

- collegamento statico
- collegamento dinamico

La differenza tra i due sta nel momento in cui viene effettuato il collegamento.

Binding statico

Il binding statico, o collegamento statico, è il processo di associazione tra elementi di un programma che avviene durante la fase di compilazione.

In questo caso, il compilatore crea l'associazione tra variabili e il loro tipo o tra costanti e il loro valore, e tale collegamento rimane invariato durante l'esecuzione del programma.

Nello specifico, il binding statico consente di collegare:

- **variabili e il loro tipo:** durante la compilazione, il compilatore associa ogni variabile al suo tipo, e questa associazione rimane invariata per tutta l'esecuzione del programma.
- **Costanti e il loro valore:** Il valore di una costante viene definito e fissato durante la compilazione, e non può essere modificato successivamente.

Nel contesto del binding dinamico, con il termine *variabile* si fa riferimento anche agli **attributi** di una classe. Gli attributi sono variabili associate a un oggetto e il loro tipo viene definito durante la compilazione. Questo collegamento tra attributi e il loro tipo è un esempio di binding statico.

Nella Programmazione Orientata agli Oggetti, il binding statico, oltre ai casi precedentemente menzionati, si applica a:

- metodi **statici (static)**: appartengono alla classe, non alle istanze. L'invocazione di questi metodi è risolta direttamente dal compilatore.
- Metodi **final**: non possono essere sovrascritti, quindi il comportamento è già definito a tempo di compilazione.
- **Overloading**: se una classe a più metodi con lo stesso nome, ma firme diverse, il compilatore sceglie il giusto metodo in fase di compilazione.

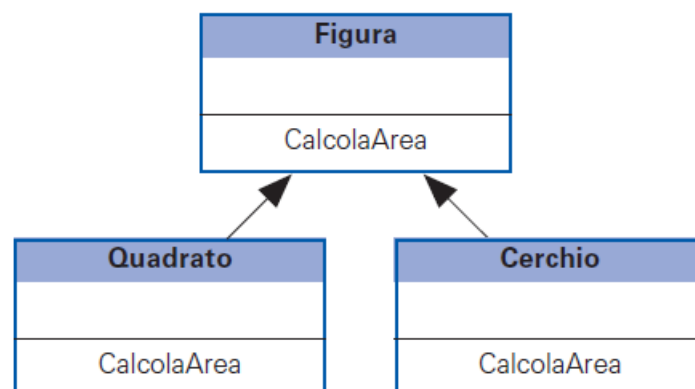
Questi collegamenti sono definiti in modo permanente e non cambiano durante l'esecuzione del programma.

Il binding statico è possibile solo se tutte le informazioni necessarie sono disponibili al momento della compilazione. Ad esempio, il tipo di una variabile viene definito e fissato durante la compilazione, garantendo maggiore efficienza e prevedibilità, ma meno flessibilità rispetto al binding dinamico.

Binding dinamico

Il binding dinamico, o collegamento dinamico, è il processo di associazione tra elementi di un programma (come metodi e oggetti) che avviene durante l'esecuzione del programma (run-time). È utile perché consente di gestire situazioni in cui le informazioni necessarie per il collegamento non sono disponibili al momento della compilazione.

Nei linguaggi orientati agli oggetti, il **binding dinamico è fondamentale per implementare il polimorfismo**. Ad esempio, quando un metodo è stato sovrascritto nelle sottoclassi (overriding), è solo durante l'esecuzione (run-time) che l'**interprete** può scegliere e collegare il metodo corretto in base all'oggetto che lo ha invocato. Questo garantisce flessibilità e adattabilità del programma durante l'esecuzione.



Nello schema si vede che la classe **Figura** è la superclasse delle classi **Quadrato** e **Cerchio**. In entrambe le sottoclassi è stato ridefinito il metodo **calcolaArea**.

L'oggetto viene creato come istanza della classe **Figura** con la seguente dichiarazione:

```
Figura f = new Figura();
```

Nel corso del programma viene invocato il metodo **calcolaArea** con la seguente istruzione:

```
f.calcolaArea();
```

Durante l'esecuzione si potrebbe verificare che l'oggetto `f` rappresenti un quadrato oppure un cerchio e, pertanto, risulta necessario stabilire quale sia il metodo corretto da eseguire: quello definito nella classe `Figura`, in `Quadrato` o in `Cerchio`? Ciò che deve essere trovato è il collegamento tra l'oggetto e il metodo `calcolaArea` corretto. Questa scelta non può essere fatta al momento della compilazione, ma durante l'esecuzione attraverso un **collegamento dinamico**.

Esempio di binding dinamico

```
public abstract class Dispositivo {
    abstract void accendi();
}

public class Televisore extends Dispositivo {
    @Override
    void accendi() {
        System.out.println("Accendendo il televisore. Benvenuto!");
    }
}

public class Smartphone extends Dispositivo {
    @Override
    void accendi() {
        System.out.println("Accendendo lo smartphone. Caricamento del
sistema operativo...");
    }
}

public class Tablet extends Dispositivo {
    @Override
    void accendi() {
        System.out.println("Accendendo il tablet. Pronto per l'uso!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dispositivo[] dispositivi = {
            new Televisore(),
            new Smartphone(),
            new Tablet()
        };
        for (int i = 0; i < dispositivi.length; i++)
        {
            dispositivi[i].accendi();
            // Binding dinamico: metodo deciso a runtime
        }
    }
}
```

Il collegamento dinamico (binding dinamico) viene effettuato dalla Java Virtual Machine (JVM) durante l'esecuzione del programma. A differenza del binding statico, che viene deciso dal compilatore a tempo di compilazione, il binding dinamico avviene a runtime, e la JVM sceglie quale versione di un metodo invocare basandosi sul **tipo reale dell'oggetto**.

Caso 1: tipo del riferimento uguale al tipo dell'oggetto

In questo caso, il tipo del riferimento e il tipo reale dell'oggetto coincidono. Il binding dinamico non è presente perché sia il compilatore sia la JVM sanno già a tempo di compilazione quale metodo chiamare. Non c'è bisogno di risolvere quale versione del metodo invocare durante l'esecuzione, dato che il tipo dell'oggetto e del riferimento sono esattamente lo stesso.

```
public class Cane {
    void abbaia() {
        System.out.println("Woof Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Cane cane = new Cane(); // Tipo riferimento = Tipo reale
        cane.abbaia(); // Binding statico a tempo di compilazione
    }
}
```

Caso 2: Tipo del riferimento è della superclasse (polimorfismo)

Quando il tipo del riferimento è della superclasse, ma il tipo reale dell'oggetto è una sottoclasse, il metodo viene scelto a runtime grazie al binding dinamico.

```
public class Animale {
    void suono() {
        System.out.println("Suono generico di un animale");
    }
}

public class Cane extends Animale {
    @Override
    void suono() {
        System.out.println("Il cane abbaia: Woof Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animale animale = new Cane();
        animale.suono();
    }
}
```

Caso 3: Tipo del riferimento è un'interfaccia

Se il tipo del riferimento è un'interfaccia, il metodo chiamato dipenderà dal tipo dell'oggetto reale. Anche qui vi è il binding dinamico.

```
public interface Dispositivo {
    public void accendi();
}

public class Smartphone implements Dispositivo {
    @Override
    public void accendi() {
        System.out.println("Accendendo lo smartphone...");
    }
}

public class Televisore implements Dispositivo {
    @Override
    public void accendi() {
        System.out.println("Accendendo il televisore...");
    }
}

public class Main {
    public static void main(String[] args) {
        Dispositivo dispositivo1 = new Smartphone();
        Dispositivo dispositivo2 = new Televisore();

        dispositivo1.accendi(); // Output: Accendendo lo smartphone...
        dispositivo2.accendi(); // Output: Accendendo il televisore...
    }
}
```